

9 – Handling Files

The programs that we developed up to now were neither able to produce permanent output nor were they able to read data inputs other than from the keyboard. Using files you can save your output data permanently and retrieve them later.

A file in general is a collection of related *records*, such as student information, marks obtained in an exam, employee salaries, etc. Each record is a collection of related items called *fields*, such as “student name”, “date of birth”, “subjects registered”, etc. The common operations associated with a file are:

- Read from a file (input)
- Write to a file (output)
- Append to a file (write to the end of a file)
- Update a file (modifying any location within the file)

In C language data is transferred to and from a file in three ways:

- Record input/output (one record at a time)
- String input/output (one string at a time)
- Character input/output (one character at a time)

9.1 The File Protocol

Accessing a file is a three step process:

- Opening a connection to a file
- Reading/writing data from/to the file
- Closing the connection

9.1.1 Opening a Connection to a File

In order to use a file on a disk you must establish a connection with it. A connection can be established using the **fopen** function. The function takes the general form:

fopen(file_name, access_mode)

The **file_name** is the name of the file to be accessed and it may also include the path. The **access_mode** defines whether the file is open for reading, writing or appending data. Table 9.1 summarises the access modes supported by **fopen** function.

Table 9.1 – File access modes

Access mode	Description
“r”	Open an existing file for reading only.
“w”	Open a file for writing only. If the file does not exist create a new one. If the file exists it will be overwritten.
“a”	Open a file for appending only. If the file does not exist create a new one. New data will be added to the end of the file.
“r+”	Open an existing file for reading and writing
“w+”	Open a new file for reading and writing
“a+”	Open a file for reading and appending. If the file does not exist create a new one.

The following code opens a file named “my file.txt” in the current directory for appending data:

```
FILE *fp;  
fp = fopen("my file.txt", "a");
```

The function **fopen** returns a pointer (referred as the *file pointer*) to the structure⁶ **FILE** which is defined in the **stdio.h** header file. When you open a file it would be better to make sure that the

⁶ Structures are used to store records with different data types.

operation is successful. If the establishment of a connection is successful the function returns a pointer to the file. If an error is encountered while establishing a connection the functions returns **NULL**.

9.1.2 Closing the Connection to a File

After a connection to a file is established it can be used to read or write data. When all the file processing is over the connection should be closed. Closing the connection is important as it writes any remaining data in the buffer to the output file. The function **fclose** is used to close the file. For example:

```
fclose(fp)
```

When closing the file the file pointer “**fp**” is used as an argument of the function. When a file is successfully closed the function **fclose** returns a **zero** and any other value indicates an error.

9.1.3 Reading Data from a File

When reading data from a ASCII file you can either read one character or one string at a time.

Reading Characters from a File

To read one character at a time you can use the **getc** function. It takes the form:

getc(file_pointer)

You can assign the output of the function **getc** to an **int** or **char** variable.

Example 9.1 – Write a program to read the file “my file.txt” which has the message:

```
Hello World!
This is my first file
```

The following program reads the file “my file.txt” one character at a time and displays it on the screen.

```
/* Program-9.1      */
#include <stdio.h>

int main()
{
    FILE *fp;
    char c;

    fp = fopen("my text.txt", "r"); //open read-only
    if(fp != NULL)
    {
        c = getc(fp);           //read the 1st character
        while ( c != EOF)      //if not the end of file
        {
            printf("%c",c);
            c= getc(fp);        //read next character
        }
        fclose(fp);           //close the file
    }
    else
        printf("\nError while opening file...");

    return 0;
}
```

Execution of Program-9.1 will display

```
Hello World!
This is my first file
```

In Program-9.1 a connection is first established to the file. Then the expression **if(fp != NULL)** evaluates whether the connection is successful. If it is not successful the program will display the message “**Error while opening file...**” If it is successful it reads the first character from the file. If the character is not the end of the file (indicated by the End Of File (**EOF**) mark) it displays the

character. Then the program continues to read the rest of the characters in the file until it finds the **EOF** mark. Afterwards the connection to the file is closed using the **fclose** function.

Reading a String from a File

In real-life applications it is more useful to read one string at a time rather than one character. With every read, the program has to check for the line feed (LF) character so it can find the end of each string. Also it must check for the EOF mark which comes at the end of the file. The **fgets** function can be used to read a string at a time. The function generally takes the form:

fgets(string, max_characters, file_pointer)

The “**string**” is a character array (also called a character *buffer*) and “**max_characters**” define the maximum number of characters to read from a line. The function **fgets** returns a **char** pointer. It returns **NULL** if **EOF** mark is encountered. One deficiency in **fgets** is that it can only read to a fixed character buffer, therefore you need to know in advance the maximum number of characters in a string.

Example 9.2 – Modify Program-9.1 such that it uses the fgets function instead of fgetc function.

Suppose the file does not have more than 100 characters in a line.

```
/* Program-9.2      */
#include <stdio.h>

int main()
{
    FILE *fp;
    char buffer[100];    //char array with 100 elements
    char *result;        // hold the result of the fgets function

    fp = fopen("my text.txt", "r"); //open read-only
    if(fp != NULL)
    {
        result = fgets(buffer, 100, fp); //read the 1st string
        while(result != NULL)           //if not the end of file
        {
            printf("%s",buffer);
            result = fgets(buffer, 100, fp); //read the next string
        }
        fclose(fp);                      //close the file
    }
    else
        printf("\nError while opening file");

    return 0;
}
```

9.1.4 Writing Data to a File

You can also write data to file either one character at a time or a string at a time.

Writing Character to a File

To write a character to a file the **putc** function can be used. It has the form:

putc(c, fp)

where **c** is the character while **fp** is the file pointer. It returns an **int** value which indicates the success or the failure of the function. It returns the **int** value of the character if it is successful, if not it returns **EOF** mark.

Example 9.3 – Write a C program to store the message “Introduction C Programming” in a file named “message.txt”.

Program-9.3 is an implementation of the above requirement. The function **putc** is used to write characters in the message to the file. To find the number of characters in the message the **strlen**

function which returns the number of characters in a string is used. A pointer is used to point to the string and the pointer is incremented by one memory location at a time.

```

/* Program-9.3      */
#include <stdio.h>
#include <string.h>

int main()
{
    FILE *fp;
    int i;           //loop counter
    char *message;
    message = "Introduction C Programming";

    fp = fopen("c:\\message.txt", "w"); //open for writing
    if (fp != NULL)                    //if success
    {
        for (i = 0 ; i < strlen(message); i++)
            putc(*(message+i), fp); //write character pointed by pointer
        fclose(fp);                //close the file
    }
    else
        printf("\nError while opening file");

    return 0;
}

```

Writing a String to a File

The advantage of **putc** is that it allows you to control every byte that you write into the file. However sometimes you may want to write one string at a time. Two functions, namely **fputs** and **fprintf** can be used for this purpose. The **fprintf** function is identical to the **printf** function only difference being that it writes to a file rather than to the screen. The format of each function is:

fputs(string, file_pointer)
fprintf(file_pointer, "%s", string)

Exercise 9.1 – Modify Program-9.3 such that it uses the **fputs** rather than the **putc** function to write the message to the file.

Exercise 9.2 – Develop a simple telephone directory which saves your friends contact information in a file named `directory.txt`. The program should have a menu similar to the following:

```

-----Menu-----
1. Add new friend.
2. Display contact info.
3. Exit
-----
Enter menu number:

```

When you press “1” it should request you to enter following data:

```

-----New friend info-----
Name : Saman
Phone-No: 011-2123456
e-Mail : saman@cse.mrt.ac.lk

```

After adding new contact information it should again display the menu. When you press “2” it should display all the contact information stored in the `directory.txt` file as follows:

```

-----Contact info-----
Name      Tel-No      e-Mail
Kamala    077-7123123   kamala@yahoo.com
Kalani    033-4100101   kalani@gmail.com
Saman     011-2123456   saman@cse.mrt.ac.lk
-----

```

Annex A – Lab 2 & 3

Introduction to C Programming

Objectives

- Compile/link C programs on Linux using gcc.
- Correct/fix syntax errors and bugs taking clues from compiler error messages and warnings.
- Debug programs through program testing.
- Learn how to detect programming errors through program testing.

Prerequisites

- Students are expected to be familiar with a text editor on Linux (such as vi or vim).
- A basic understanding of compiling and linking in concept.
- Knowledge in area such as C language syntax, input output function is recommended.

Requirements

- The GNU C and C++ compiler⁷.

Remarks

- This assignment will be continued next week as well.
- No evaluation will be done based on these lab sessions.
- All the source codes and executables related to this lab session should be saved in a folder named **Lab2_3** which should be created within your **home** directory.

A.1 Compiling and Linking

In *compiler based* programming languages like C, the *source code* must be first *compiled* and then *linked* with the necessary *libraries* to produce an *executable* program. The process of compiling and linking depends on the environment in which it is carried out, the particular compiler or the linker involved and tools used if any. *Integrated Development Environments* (IDEs) are software tools which provide facilities for; editing, compiling, linking and debugging software (will cover in Lab5).

In this section we shall learn how to compile and link a C program using the GNU C and C++ compiler, **gcc**. No other tools will be used.

When you invoke **gcc**, it normally does pre-processing, compilation, assembly and linking. The overall options allow you to stop this process at an intermediate stage. For example, the **-c** option says not to run the linker. Then the output consists of *object* files output by the *assembler*.

Step 1: Type the C program given below using text editor such as **Kwrite** and then save it as **test1.c** in your **home** directory.

Start **Kwrite** as follow; **Main Menu → Accessories → More Accessories → Kwrite**

```
/*display Hello World */           //Comments
#include <stdio.h>                   //Pre-processor directives

int main()                          //Start main function
{
    printf("Hello world!\n"); //use printf function to display message
    return 0;
}                                   //End main
```

Step 2: Open a new terminal with the shell prompt by **right-clicking** on the **Desktop** area and selecting "**new Terminal**". Now you can enter any Linux shell command using the shell. Go to the **Lab2_3** directory where you saved the program source file **test1.c** using **cd** command.

⁷ <http://www.gnu.org/>